PROJECT REPORT

CS:ETL Pipeline with Machine Learning Integration

Automated retail data ingestion, cleaning, prediction, and database storage

Prepared by	Rishi Kumar Jaiswal
Project type	Data Engineering / ETL with Machine Learning
Primary dataset	Online Retail II (online_retail_II.csv)
Execution schedule	Designed to run every 10 minutes using Apache Airflow

Executive Summary

This project demonstrates an automated data pipeline that prepares retail transaction data for analysis and generates machine-learning predictions. Apache Airflow orchestrates the workflow, Python and pandas handle data cleaning and preparation, PostgreSQL stores processed records and prediction outputs, and a scikit-learn regression model predicts retail item quantities.

The workflow is configured to run every 10 minutes. The ETL task is defined to run before the prediction task, so predictions are generated after the cleaning and loading step completes successfully. The Airflow interface screenshot reviewed during project verification showed successful ETL and prediction task runs, alongside a scheduler heartbeat warning and a failed run in the displayed history. These operational details are recorded transparently rather than treating every run as successful.

Project objective

Build a repeatable workflow that extracts retail data, cleans it, stores it in PostgreSQL, and generates quantity predictions using a trained machine-learning model.

Contents

1. Introduction
2. Objectives and Scope
3. Technology Stack
4. System Architecture and Workflow
5. Dataset and Data Preparation
6. ETL Implementation
7. Machine Learning Integration
8. Database Design
9. Airflow Orchestration
10. Testing and Observed Results
11. Limitations and Future Improvements
12. Conclusion

Appendix A. Project Structure

1. Introduction

Organizations often receive raw data that contains missing values, inconsistent formats, duplicate records, and invalid values. Before this data can be used reliably, it needs to be processed in a consistent and repeatable way. An Extract, Transform, Load (ETL) pipeline automates this work.

This project applies ETL concepts to retail transaction data and extends the workflow with machine learning. The pipeline prepares the data, stores cleaned records in a relational database, and uses a trained model to estimate item quantities from transaction-related features.

2. Objectives and Scope

- Ingest the Online Retail II CSV dataset from the project data directory.
- Clean and standardize important retail transaction fields using Python and pandas.
- Persist cleaned transaction records in PostgreSQL.
- Train a regression model to predict item quantity using historical retail features.
- Store model predictions in PostgreSQL for later inspection and analysis.
- Use Apache Airflow to schedule and monitor the workflow at 10-minute intervals.
- Apply basic reliability measures such as task retries, execution timeouts, and duplicate-conflict handling.

Scope note: the project is a practical demonstration. Production deployment would require additional monitoring, data-quality checks, model-governance controls, and operational testing.

3. Technology Stack

Technology	Role in the project	Why it is used
Apache Airflow	Workflow orchestration	Schedules tasks, tracks runs, and manages task dependencies.
Python	Pipeline and model code	Provides the main language for transformation and prediction

		logic.
pandas	Data preparation	Reads CSV data and cleans, filters, and reshapes transaction records.
PostgreSQL	Persistent storage	Stores cleaned retail data and prediction results in relational tables.
SQLAlchemy	Database access	Provides database connections and transaction handling from Python.
scikit-learn	Machine learning	Builds preprocessing steps and the Random Forest regression model.
joblib	Model persistence	Saves and loads the trained model artifact.
Docker Compose	Local service environment	Runs the Airflow and PostgreSQL services as containers.

4. System Architecture and Workflow

The main workflow is designed as a sequence of dependent stages:

1. Source: read the online_retail_II.csv file from the Airflow project data directory.
2. Transform: standardize text, parse dates and numeric fields, remove duplicates and invalid records, and prepare consistent column names.
3. Load: save the cleaned CSV output and insert eligible records into the cleaned_retail PostgreSQL table.
4. Predict: load the saved machine-learning model, prepare features from eligible cleaned rows, and generate quantity predictions.
5. Store results: insert prediction records into the retail_predictions PostgreSQL table.
6. Orchestrate: Airflow schedules the ETL task and runs the prediction task after ETL succeeds.

Workflow dependency
run_retail_etl → run_retail_predictions. The prediction task is downstream of ETL, which helps ensure it uses cleaned data.

5. Dataset and Data Preparation

The project uses the Online Retail II dataset supplied as online_retail_II.csv. The dataset contains retail transaction information such as invoice identifiers, product codes and descriptions, quantities, invoice dates, prices, customer identifiers, and countries.

5.1 Fields used

Source field	Meaning
Invoice	Transaction or invoice identifier
StockCode	Product identifier
Description	Product description

Quantity	Number of units recorded for a transaction line
InvoiceDate	Transaction date and time
Price	Unit price
Customer ID	Customer identifier, where available
Country	Country associated with the transaction

5.2 Cleaning rules

- Trim surrounding whitespace from relevant text fields.
- Convert invoice dates to datetime values and convert quantity and price to numeric values; invalid conversions become missing values for filtering.
- Remove exact duplicate rows.
- Remove rows missing essential transaction fields.
- Keep records with positive quantity and positive price.
- Exclude empty invoice, product-code, or country values and retain whole-number quantities.
- Rename columns to lowercase snake_case names for consistent use in code and SQL.

Data-quality note
For invoice groups with missing customer IDs, the current ETL code generates placeholder customer identifiers. These generated IDs are not verified real customer IDs and should not be interpreted as recovered customer information.

6. ETL Implementation

The ETL logic is implemented in src/etl.py. It reads the raw CSV, applies the cleaning rules, writes a cleaned CSV file named cleaned_retail.csv, and loads cleaned rows into PostgreSQL.

6.1 Database loading approach

- The database connection is read from the APP_DATABASE_URL environment variable rather than being hard-coded in the script.
- The cleaned_retail table is created if it does not already exist.
- Rows are staged and inserted in batches of 5,000 records to handle a large dataset more efficiently.
- The load uses a unique index and ON CONFLICT DO NOTHING to reduce repeated inserts for records matching the configured uniqueness key.
- Database changes are handled inside a transaction so the operation can be committed or rolled back as a unit.

The current unique key uses invoice, stock code, invoice date, quantity, and price. If two legitimate transaction lines share all of these values, they may be treated as a conflict. A production version should use a stable source row identifier or a more reliable business key where available.

7. Machine Learning Integration

The model training code is located in models/train_model.py, and prediction execution is implemented in src/predict.py. The project uses a RandomForestRegressor to predict transaction quantity.

7.1 Prediction target and features

Type	Fields
Target	quantity
Numeric features	price, month, day_of_week, hour
Categorical features	stock_code, country

7.2 Training process

7. Read eligible cleaned records from PostgreSQL, with a maximum training sample of 200,000 rows in the current script.
8. Order records by invoice date and use a chronological 80/20 training/test split.
9. Impute missing numeric values using the median and categorical values using the most frequent value.
10. One-hot encode categorical features, allowing unknown categories and grouping infrequent categories.
11. Train a Random Forest regression model with 100 trees, maximum depth 12, minimum leaf size 2, and a fixed random seed.
12. Calculate Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) on the test split and save the trained model as retail_quantity_model.joblib.

The model file is loaded by the prediction task. Prediction records include the source row identifier, invoice, stock code, invoice date, actual quantity, predicted quantity, and model version.

7.3 Prediction safeguards and limitations

- The prediction script checks that the trained model file exists before attempting predictions.
- Prediction work is processed in batches of 5,000 rows.
- The database insert is configured to avoid duplicate prediction inserts for records matching its conflict rules.
- The current prediction de-duplication logic also compares invoice, stock code, and invoice date. Different lines for the same product and invoice date may therefore be skipped in some cases.
- No specific MAE or RMSE score is reported here because a verified model-training output was not available for this report.

8. Database Design

8.1 cleaned_retail

This table stores cleaned transaction data. Its fields include an internal row ID, invoice, stock code, description, quantity, invoice date, price, customer ID, country, and load timestamp.

8.2 retail_predictions

This table stores model outputs linked to source transaction rows where possible. Fields used by the prediction logic include source_id, invoice, stock_code, invoice_date, actual_quantity, predicted_quantity, and model_version.

The database connection is configured through APP_DATABASE_URL. Credentials should be supplied through local environment configuration and must not be included in the submitted source-code archive or report.

9. Airflow Orchestration

Configuration	Current project setting
DAG ID	cs_etl_pipeline
Schedule	*/10 * * * * (every 10 minutes)
Catchup	False
Maximum active runs	1
Retries	2 retries
Retry delay	2 minutes
Task timeout	1 hour per task
Task order	run_retail_etl → run_retail_predictions

The DAG definition is stored in dags/etl_pipeline.py. Airflow provides a visual run history and task-level logs, which help operators understand whether tasks completed or failed.

10. Testing and Observed Results

During verification, Docker Compose showed the Airflow webserver, Airflow scheduler, and PostgreSQL services as running, with PostgreSQL marked healthy. The Airflow interface displayed the cs_etl_pipeline DAG as enabled and showed successful ETL and prediction task runs.

The same Airflow screen also displayed a scheduler heartbeat warning and a failed task instance in the run history. These should be treated as operational items to investigate before a production deployment. The project demonstrates successful task execution, but the screenshot does not establish that every scheduled run succeeds.

Check	Observed evidence	Interpretation
Docker services	Airflow, scheduler, PostgreSQL listed as Up; PostgreSQL healthy	Services were running at the time checked.
DAG schedule	cs_etl_pipeline enabled; every 10 minutes	Recurring schedule is configured.
ETL task	Multiple green task instances visible	Successful ETL runs are present in history.
Prediction task	Selected task instance status: success	At least one prediction task run completed successfully.
Scheduler health	Heartbeat warning displayed	Further operational verification is recommended.

11. Limitations and Future Improvements

- Resolve and monitor scheduler heartbeat warnings; configure an alert for missed or failed scheduled runs.
- Improve row-level idempotency using a stable source record identifier, and review unique keys for both ETL and prediction tables.

- Replace generated customer IDs with explicit missing-value handling or a clearly documented surrogate-key strategy.
- Record row counts at each stage and add automated data-quality checks for missing values, invalid quantities, and duplicate rates.
- Track model metrics over time, compare the Random Forest against a simple baseline, and document training data dates and model versions.
- Add unit and integration tests for data cleaning, database inserts, prediction output, and repeat-run behavior.
- Use secrets management for database credentials and avoid including credentials, local logs, or large raw data files in the submission archive.
- Consider incremental ingestion using source timestamps or a reliable watermark rather than repeatedly scanning the full source file.

12. Conclusion

The CS:ETL Pipeline with Machine Learning Integration project brings together core data engineering and machine learning components in one automated workflow. Python and pandas prepare retail transaction data, PostgreSQL stores cleaned records and predictions, scikit-learn provides a quantity prediction model, and Apache Airflow coordinates execution on a 10-minute schedule.

The reviewed Airflow evidence shows successful ETL and prediction task runs. The scheduler warning and failed run in the history are documented as limitations to address before production use. Overall, the project provides a practical foundation for automated data processing and machine-learning integration.

Appendix A. Project Structure

```
cs_etl_ml/
├── dags/
│   └── etl_pipeline.py
├── data/
│   ├── online_retail_II.csv
│   └── cleaned_retail.csv
├── models/
│   ├── train_model.py
│   └── retail_quantity_model.joblib
├── src/
│   ├── etl.py
│   ├── inspect_data.py
│   └── predict.py
├── tests/
├── plugins/
├── docker-compose.yaml
├── Dockerfile.airflow
└── check_etl_files.py
```

Appendix B. Submission Checklist

- Project report (this document).
- Source-code ZIP containing the DAG, Python scripts, Docker files, and README.
- Airflow screenshot showing the DAG and task history.
- Database/table evidence and model evaluation output, if available.
- Exclude .env files, passwords, database credentials, virtual environments, Docker volumes, and other secrets.